

PyTorch Documentation Assistant

A Retrieval-Augmented Generation (RAG) System

Project Report

Author: Abdullah Khan

Abstract

This project implements a production-quality Retrieval-Augmented Generation (RAG) assistant that answers PyTorch questions using only the official library documentation and cites its sources. Documentation is ingested directly from the installed package's docstrings across all publicly documented modules, split into structure-aware chunks, embedded with a local sentence-transformer model, and stored in a Chroma vector database. Queries are served by a two-stage retriever (bi-encoder search followed by cross-encoder reranking) and answered by a locally hosted, 4-bit quantized Qwen2.5-7B-Instruct model constrained to the retrieved context. The system is exposed through a Gradio web interface. This report explains and justifies every major design decision and documents the retrieval failures encountered and resolved during development.

1. Introduction and Problem Statement

Large Language Models (LLMs) are powerful at answering technical questions, but when used on their own they have two failure modes that are unacceptable for a documentation assistant. First, their knowledge is frozen at training time, so it can be out of date relative to the current version of a fast-moving library like PyTorch. Second, and more seriously, they hallucinate: when unsure, an LLM will often invent a function name, argument, or behaviour that does not exist, and state it with full confidence. For a developer relying on the answer to write real code, a confidently wrong API reference is worse than no answer at all.

Retrieval-Augmented Generation (RAG) addresses both problems. Instead of trusting the model's memory, a RAG system retrieves relevant passages from a trusted knowledge source at query time and instructs the model to answer using only those passages, citing them. This grounds every answer in verifiable documentation and lets the system honestly say “I don't have information on that” when the answer is not in its sources — which is far safer than fabricating one.

Problem Statement

The goal of this project is to build a RAG assistant for the PyTorch library that: ingests PyTorch documentation; converts it into searchable embeddings; retrieves the most relevant passages for a user's question; generates an answer grounded strictly in the retrieved context; cites the source documentation; and exposes all of this through a simple user interface. A central requirement is that every major design decision — the LLM, the embedding model, the vector database, and the chunking strategy — must be deliberately chosen and fully justified, and defensible under questioning.

Summary of the System

The table below summarises the stack. Each choice is justified in detail in the sections that follow.

Decision	Choice	Primary reason
Document source	Package docstrings	Authoritative source the official docs are built from; no web scraping
LLM	Qwen2.5-7B-Instruct (4-bit)	Strong grounded QA, permissive license, fits a free 16 GB GPU
Embedding model	BAAI/bge-small-en-v1.5	Beats the common default on retrieval; small, CPU-friendly, offline
Vector database	Chroma (cosine / HNSW)	Zero-setup, stores text + metadata + vectors together, persistent
Retrieval	Bi-encoder + cross-encoder rerank	Two-stage retrieval for precision on a large, technical corpus
Chunk size	500 tokens, 50 overlap	Fits the embedder's 512-token limit; keeps docstring sections intact
Interface	Gradio	One-line public share link from Google Colab; no tunneling needed

2. Which LLM Did I Choose? (and Why)

Choice: Qwen2.5-7B-Instruct, run locally with 4-bit quantization via the HuggingFace Transformers library on a Google Colab T4 GPU.

Reasoning

- **Grounded question-answering ability.** Qwen2.5-7B-Instruct is an instruction-tuned model with strong reasoning and instruction-following, which is exactly what RAG needs: the model must obey the constraint “answer only from this context and cite it” rather than free-associating from memory.
- **Permissive licensing.** It is released under the Apache 2.0 license, so it can be used without the gated-access approval required by some alternatives (e.g. Llama). This keeps the project fully self-contained.
- **Fits a free GPU.** At full 16-bit precision a 7-billion-parameter model needs roughly 14–16 GB of VRAM. Using 4-bit NF4 quantization (via the bitsandbytes library) shrinks it to about 5 GB, so it fits comfortably on Colab's free T4 (16 GB) with room for the KV cache. Weights are stored in 4 bits but computation runs in 16-bit for accuracy — a deliberate storage/accuracy trade-off.
- **Fully local and free.** Running the model locally (rather than calling a paid API) keeps the whole system offline and cost-free, and means no user data leaves the machine.

Alternatives Considered

Option	Why not chosen
GPT-4 / Claude (API)	Excellent quality but paid, require an API key, and send data to a third party — breaks the free/offline design.
Llama-3-8B-Instruct	Comparable quality but gated license requiring access approval; Qwen avoids that friction.
A smaller 1–3B model	Would run faster but follows the strict “answer only from context” instruction less reliably, risking ungrounded answers.

Generation settings. The model is run at a low temperature (0.1) on purpose: a documentation assistant should be factual and deterministic, not creative. This makes answers stable and closely tied to the retrieved text.

3. Which Embedding Model? (and Why)

Choice: BAAI/bge-small-en-v1.5, a 33-million-parameter sentence-transformer producing 384-dimensional embeddings, run locally.

An embedding model converts text into a vector of numbers such that texts with similar meaning end up close together in vector space. This is what makes retrieval semantic — it matches on meaning rather than exact keywords, so a question like “how do I turn off gradient tracking” can find the documentation for `torch.no_grad` even if it never uses those exact words. Retrieval quality is upstream of everything else in a RAG system: if the right passage is never retrieved, no amount of clever prompting can recover it.

Reasoning

- **Better than the common default.** The usual default for local RAG is all-MiniLM-L6-v2. BGE-small is the same size class and speed but scores higher on the MTEB retrieval benchmark, so it is a free quality upgrade over the obvious choice.
- **Small, fast, CPU-friendly, offline.** At 33M parameters and 384 dimensions it embeds the corpus quickly and runs without a GPU or an API. The compact 384-dimensional vectors also make similarity search fast and storage small.
- **Correct query/document handling.** BGE models are trained with an asymmetry: queries are prefixed with a short instruction (“Represent this sentence for searching relevant passages:”) while documents are embedded without it. This measurably improves retrieval and is implemented faithfully in the code. Embeddings are also normalised to unit length so that cosine similarity reduces to a fast, correct dot product.

Alternatives Considered

Option	Why not chosen
Keyword search (TF-IDF / BM25)	Matches words, not meaning — “automobile” would not match “car”. Insufficient alone for varied phrasing.
all-MiniLM-L6-v2	Solid and popular, but BGE-small matches its cost while scoring better on retrieval.
bge-base / bge-large	More accurate but several times slower/heavier — unnecessary for this corpus size.
OpenAI text-embedding-3	Excellent, but costs money, needs a key, and sends data off-machine.

4. Which Vector Database? (and Why)

Choice: Chroma, run in persistent mode, configured with cosine distance over an HNSW index.

Reasoning

- **Zero server setup.** Chroma runs embedded inside the Python process — it is simply pip-installed, with no separate server or Docker container to manage (unlike Weaviate, Milvus, or Qdrant). This is appropriate for the project's scale and timeline.
- **Stores everything together.** Chroma holds each chunk's vector, its original text, and its metadata (the source name used for citation) in one place. When a search returns a match, the text and citation come with it — no separate bookkeeping. This directly serves the citation requirement.
- **Persistent.** Using a persistent client writes the index to disk, so embedding is a one-time cost and the index can be reused across sessions.
- **Deliberate configuration.** Cosine distance is set explicitly (Chroma defaults to L2). With unit-normalised embeddings, cosine measures pure semantic direction and is the standard pairing for text retrieval. The underlying HNSW index is an approximate-nearest-neighbour structure that keeps search fast as the corpus grows — overkill at this scale, but it means the design scales without changes.

Alternatives Considered

Option	Why not chosen
Plain Python list + brute force	Works at small scale but hand-rolls persistence, text storage and metadata; not scalable or clean.
FAISS	Very fast, but stores only raw vectors — no text, metadata, or built-in persistence, all of which would have to be added manually.
Pinecone / Weaviate Cloud	Managed and scalable, but need accounts/API keys and a cloud service — unjustified overhead for this corpus size.

5. Chunk Size (and Why)

Choice: A target of 500 tokens per chunk with 50 tokens of overlap (~10%), using a structure-aware splitter that breaks on paragraph boundaries and prepends a signature header to each piece of a split docstring.

Why chunk at all

An embedding is a single fixed-size vector regardless of how much text it encodes. If an entire long docstring is embedded as one vector, the result is a blurred average in which no single detail is findable — like blending many ingredients into one smoothie where no individual flavour stands out. Splitting the text into smaller chunks produces sharp, specific vectors, improves retrieval precision, and keeps each retrieved passage small enough to fit several into the LLM's context window at once.

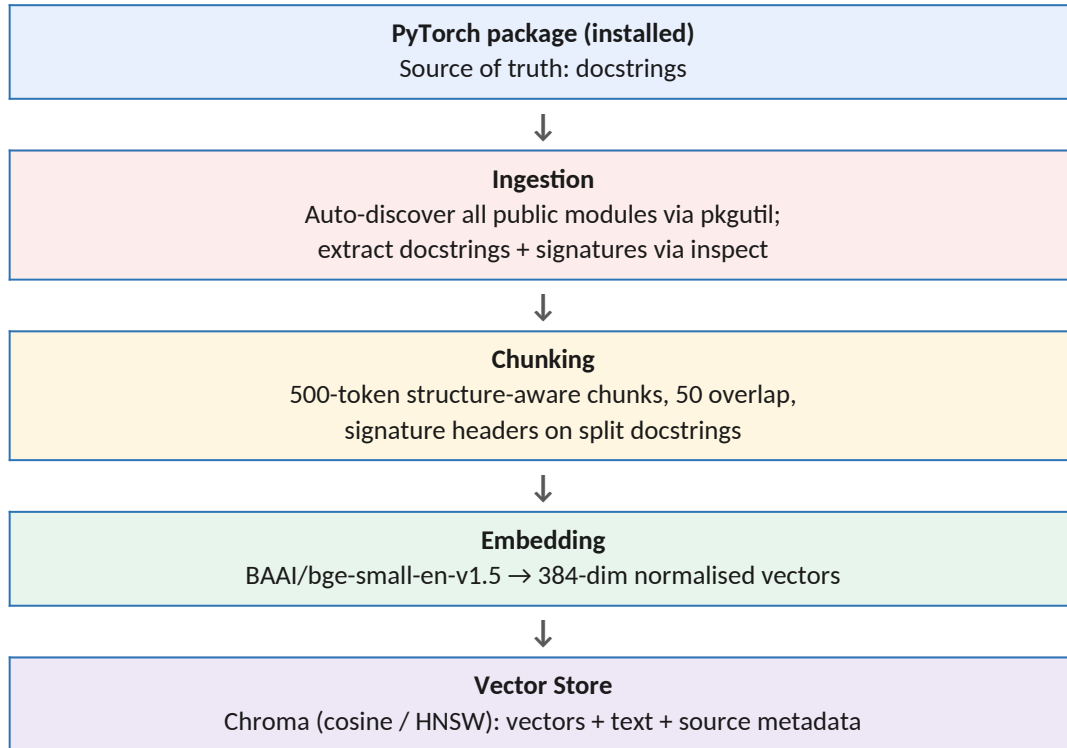
Why 500 tokens and 50 overlap

- **500 tokens.** The chosen embedding model has a hard input limit of 512 tokens; any longer text is silently truncated. Targeting 500 uses almost the full capacity while staying safely under the ceiling. Chunks are measured in tokens using the model's own tokenizer, so the counts match exactly what the model sees.
- **50 overlap (~10%).** Consecutive chunks share 50 tokens so that an idea sitting on a chunk boundary is not cut in half and lost from both chunks. Ten to fifteen percent overlap is a standard sweet spot — enough to preserve boundary context without wasteful duplication.
- **Structure-aware splitting.** Rather than cutting blindly every N characters, the splitter breaks on paragraph boundaries so that a docstring's description, its Args block, and its Example block tend to stay intact. Most docstrings are already under 500 tokens and pass through as a single chunk; only long ones are split.
- **Signature headers.** When a long docstring is split, each resulting chunk has the object's signature prepended as a short header. This is a lightweight form of contextual chunking: it ensures that even a chunk containing only a code example still identifies which function it belongs to, which materially improved retrieval (see Findings).

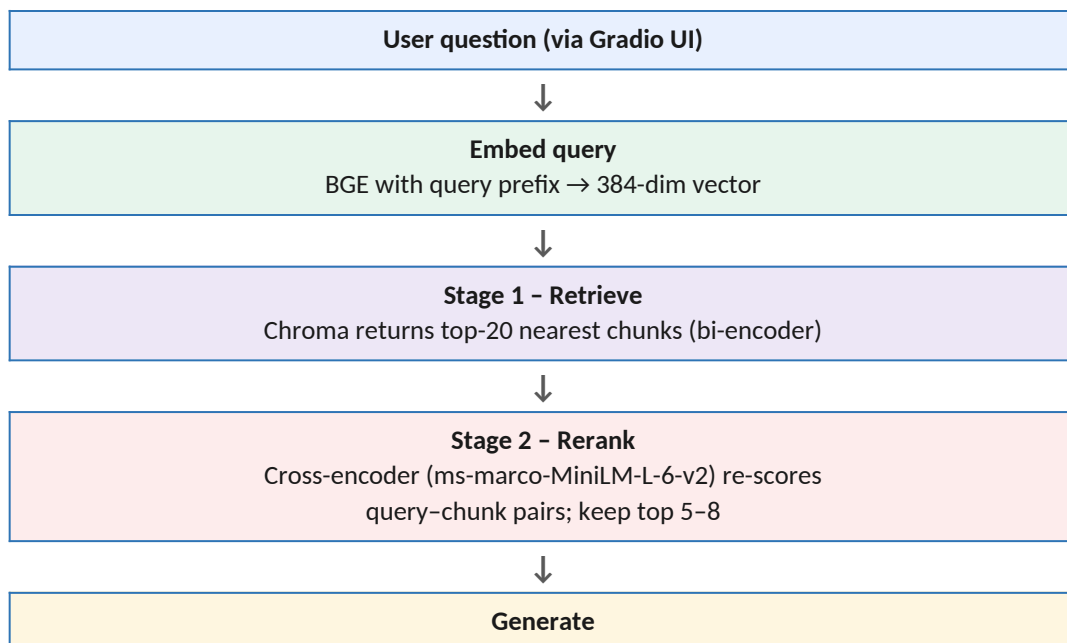
6. Architecture of the Implemented RAG System

The system has two phases. The indexing phase runs once, offline, to build the searchable database. The querying phase runs for every user question. The diagram below shows both.

Indexing Phase (run once)



Querying Phase (per question)



Qwen2.5-7B-Instruct (4-bit) answers using ONLY
retrieved context; refuses if not present



Answer + Citations
Grounded answer with [Source: ...] labels

7. Findings and Results

Development was iterative. Rather than working on the first attempt, the system exposed several real retrieval problems that were diagnosed and fixed — the process that most improved quality and understanding.

Corpus

Ingesting all publicly documented PyTorch modules (auto-discovered, excluding private/internal ones) produced 838 modules and 8,439 unique documented objects. A verification check confirmed 0% exact-text duplication across the corpus. After chunking these became 17,331 chunks (average ~315 tokens, maximum 500), of which the majority came from splitting long docstrings — concrete evidence that chunking mattered for this corpus.

Finding 1 – Naive chunking separated examples from descriptions

The first retrieval test (“How do I create a linear layer?”) failed: it returned detached code-example chunks from unrelated functions and missed `torch.nn.Linear` entirely. Diagnosis: long docstrings had been split so that Example blocks became separate chunks with no descriptive context, and a plain-English question does not match a chunk of pure code. Fix: prepend the object’s signature as a header to every split chunk (contextual chunking). This moved the target chunk from outside the top 15 into the top 5.

Finding 2 – Small embedders cluster near-synonymous technical terms

Even after the header fix, `torch.nn.Linear` ranked only 5th, with the top results packed into a very narrow similarity band. Diagnosis: the small bi-encoder correctly identified the neighbourhood of neural-network layers but could not sharply rank the exact target above near-synonyms such as `LayerNorm`. This is a known limitation of pure dense retrieval on technical vocabulary.

Finding 3 – Reranking fixed the ranking problem

Adding a cross-encoder reranker as a second stage — over-retrieve the top 20, then re-score each query-chunk pair jointly and keep the best few — resolved the issue. On the small corpus it moved `torch.nn.Linear` from rank 5 to rank 1. On the full 17,331-chunk corpus the same query still ranked the correct chunk first, now with a much wider score margin (−3.96 versus −7.26 for the next result), showing that retrieval quality held — and even sharpened — at 7.5× the corpus size.

Finding 4 – Grounding works: the system refuses out-of-scope questions

Asked “How do I bake a chocolate cake?”, the assistant correctly responded that it has no information on that in the provided documentation, rather than fabricating an answer. This demonstrates that the grounding constraint is effective — the model does not answer outside its sources.

Finding 5 – Distinguishing lookup questions from advice questions

Specific API questions (“How do I use the Adam optimizer?”) produced accurate, cited answers. A recommendation question (“What optimizer should I use?”) initially triggered an unwanted refusal, because no docstring literally states which optimizer to pick. The prompt was refined so that for “which should I choose” questions the assistant explains the available options found in the context and lets the user decide, without inventing a recommendation — remaining grounded while being genuinely helpful. Off-topic questions still correctly refuse, confirming the prompt was relaxed without weakening grounding.

Example Result

Question: “How do I create a linear layer?”

Answer (abridged): Explains that `torch.nn.Linear` applies an affine transformation $y = xA^T + b$, shows `nn.Linear(in_features, out_features, bias=True)`, and describes the arguments — all drawn from the retrieved docstring.

Citation: Sources: [`torch.nn.Linear`]

8. Conclusion

This project delivers a complete, working RAG assistant for PyTorch that meets every requirement: it ingests documentation, builds searchable embeddings, retrieves relevant context, generates answers grounded strictly in that context, cites its sources, and provides a simple web interface. Every major design decision — docstring ingestion, the BGE-small embedding model, the Chroma vector store, the 500-token structure-aware chunking, cross-encoder reranking, and the locally hosted 4-bit Qwen2.5-7B generator — was made deliberately, with alternatives considered and rejected for stated reasons.

The most valuable outcomes came from treating retrieval as something to measure and improve rather than assume. Diagnosing why a basic query failed, fixing it with contextual chunking, adding reranking, and verifying that quality held after expanding the corpus to all public modules produced a system that is both more capable and better understood than a first-attempt pipeline would have been.

Limitations and Future Work

- **Conceptual coverage.** The corpus is API-reference docstrings, so “how does X work conceptually” questions are weaker. Adding PyTorch's hand-written conceptual/tutorial pages is a clear extension.
- **Hybrid search.** Combining keyword (BM25) scoring with dense retrieval would further help exact-term technical queries, complementing the reranker.
- **Citation friendliness.** Auto-discovery surfaces canonical deep module paths (e.g. `torch.nn.modules.linear.Linear`); mapping these back to the common short aliases (`torch.nn.Linear`) would make citations friendlier.
- **Conversational memory.** Each question is currently answered independently; adding multi-turn context would allow follow-up questions.

Overall, the system demonstrates the core value of RAG: accurate, source-grounded answers on a fast-moving technical library, with honest refusal when a question falls outside its knowledge.